

フォント解決の実測

Linux と macOS

oxi / feat/pdf-font-resolution

8ef96b91 → dbb06d08 → 51355a7a → 5288e673 → 88d13003

2026-09-03

	Linux	macOS
索引された face	2,069	740
type1 != 0	24 / 50	7 / 50
composite_ok	50 / 50	50 / 50
白紙	0	0
総頁数	650	650

フォント解決の Linux 実測

`crates/oxidocs-cli/FONT_CHECKS.md` の手順を Linux 実機で通した記録。 `feat/pdf-font-resolution` を土台 `8ef96b91` から先端 `88d13003` まで4回、同じ機械で測り直したもの。3回目以降は blindC50 コーパス 50冊を使っている。

SSIM の採点は含まない（真値が Word の PDF なので Windows が要る）。ここにあるのはレンダ結果と、その構造・ink の検査。

項目	値
OS	Linux 7.0.0-30-generic #30-Ubuntu SMP PREEMPT_DYNAMIC x86_64
cargo	1.94.0
python3 / fontTools	3.14.4 / 4.63.0
システムフォント	2,226 face (Carlito・Liberation・Caladea・Noto CJK 導入済み)

Microsoft の書体は1つも無い機械なので、同梱フォントと代替の経路がそのまま出る。

Oxi docx→PDF Linux 実測結果

2026-09-02。oxi-linux-check.md の手順を Linux 実機で通した記録。

- 測ったもの: feat/pdf-font-resolution の先端 dbb06d08 (GitLab から取得)
- 土台: 8ef96b91 (同じ機械で先に一通り測って「前」の数字を取った)
- 作業ツリー: /data/m2labo/oxi-linuxcheck (8ef96b91 の別クローン。無関係な変更は混ざっていない)

同送された oxi-fontwork.tgz とブランチの Rust ソースは **CRLF を除いて完全一致**だった。差分はひとつだけ — tools/metrics/_oxipdf_native_blindC50.py は tgz にしか無く、ブランチには入っていない。

環境

項目	値
uname -a	Linux ubuntu 7.0.0-30-generic #30-Ubuntu SMP PREEMPT_DYNAMIC Fri Jul 31 18:22:54 UTC 2026 x86_64 GNU/Linux
cargo --version	cargo 1.94.0 (85eff7c80 2026-01-15) (PATH には無い。 ~/.cargo/bin/cargo)
python3	3.14.4
fontTools	4.63.0 (入っている)
システムフォント	fc-list 2,226 face。Carlito / Liberation / Caladea / Noto CJK すべて導入済み

手順書の前提が2つ外れている。fontTools は入っており、Carlito・Liberation・Caladea も ディストリのパッケージとして既にある。つまりこの機械は「何も無い Linux」ではないので、退路の検査は意図的に環境を壊して別途行った (下記 §4)。

1. 建つか — 建つ

	結果
土台 8ef96b91	Finished release profile in 1m 57s /rc=0
ブランチ dbb06d08	Finished release profile in 59.47s /rc=0 (増分ビルド)

error 0 件。新規の3ファイル (fontidx.rs / font_util.rs / writer.rs) 由来の警告も 0 件。

cargo test -p oxipdf-core:

	lib	統合	計
土台	55 passed / 0 failed	13 passed	68
ブランチ	58 passed / 0 failed	13 passed	71

新規3本すべて緑:

```
test font_util::tests::extract_ttc_face_lifts_the_named_member ... ok
test font_util::tests::lifted_face_offsets_are_self_relative ... ok
test writer::tests::test_ascii_run_with_embedded_font_is_composite ... ok
```

手順書の「56 passed」は Linux では 58 になる。土台の lib が 55 本あるため。数が合わないのは失敗ではない。

2. フォントを見つけるか — 見つける

```
faces indexed: 2069          elapsed 0.73 s
```

同梱16本がすべて同梱ディレクトリに解決した。システムにも同じ Carlito / Liberation があるが、FontIndex::build が extra を先に走査するので同梱側が勝っている（設計どおり）。

Carlito	regular	.../crates/oxidocs-cli/fonts/Carlito-Regular.ttf#0
Carlito	bold	.../crates/oxidocs-cli/fonts/Carlito-Bold.ttf#0
Carlito	italic	.../crates/oxidocs-cli/fonts/Carlito-Italic.ttf#0
Carlito	bold-italic	.../crates/oxidocs-cli/fonts/Carlito-BoldItalic.ttf#0
Liberation Sans	regular	.../fonts/LiberationSans-Regular.ttf#0
Liberation Serif	regular	.../fonts/LiberationSerif-Regular.ttf#0
Liberation Mono	regular	.../fonts/LiberationMono-Regular.ttf#0

Calibri / Cambria / Arial / Times New Roman / Courier New / Verdana / Tahoma / Segoe UI / Georgia / Wingdings / Symbol は全 ABSENT。これで正しい。

bundled_font_dirs() の弱点は、予想より厄介な形で出た

手順書は「バイナリを別の場所へコピーすると当たらない」と書いているが、実際は逆で、**当たってしまう**。env! ("CARGO_MANIFEST_DIR") はビルド時に絶対パスで焼き込まれるため、バイナリをどこへ移しても /data/m2labo/oxi-linuxcheck/crates/oxidocs-cli/fonts を読みに行く。

```
$ /somewhere/else/oxidocs fonts-index Carlito
Carlito regular /data/m2labo/oxi-linuxcheck/crates/oxidocs-cli/fonts/Carlito-Regular.ttf#0
```

配布物としては、ビルドマシンの作業ディレクトリを実行時に覗く挙動になる。そのパスが消えていれば無言で外れ、別人の別の中身が置かれていればそれを読む。

ソースツリーの fonts/ を隠して配布バイナリ相当にすると、システム側へ落ちた:

```
faces indexed: 2067
Carlito          regular /usr/share/fonts/truetype/crosextra/Carlito-Regular.ttf#0
Liberation Sans  regular /usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf#0
```

つまり Debian/Ubuntu では `fonts-crosextra-carlito` と `fonts-liberation` が同梱フォントの役割を肩代わりしてしまうので、この弱点はこの機械では露見しにくい。素のコンテナで初めて効く。

3. Python 無しで PDF が正しく出るか — 出る

通常 (fontTools あり)

`tests/fixtures/basic_test.docx` の標準エラー全文:

```
Parsed: 1 section(s), 4 blocks total
Font subset (FCalibri): OK 22 mappings, 22 widths
PostScript name: Carlito-Regular
Substituting Carlito for Calibri (not installed; advance widths match on ASCII)
Font subset (FCalibri_I): OK 9 mappings, 9 widths
PostScript name: Carlito-Italic
Substituting Carlito for Calibri (not installed; advance widths match on ASCII)
Font subset (FCalibri_B): OK 17 mappings, 17 widths
PostScript name: Carlito-Bold
Substituting Carlito for Calibri (not installed; advance widths match on ASCII)
Font subset (latin): OK 29 mappings, 29 widths
PostScript name: Carlito-Regular
Embedding Latin font (12920 bytes, 29 glyphs)
Font subset (latin-bold): OK 29 mappings, 29 widths
PostScript name: Carlito-Bold
Embedding Latin Bold font (14252 bytes, 29 glyphs)
Font subset (cambria): OK 29 mappings, 29 widths
PostScript name: Caladea-Regular
Embedding Cambria font (11332 bytes, 29 glyphs)
Font subset (cambria-bold): OK 29 mappings, 29 widths
PostScript name: Caladea-Bold
Embedding Cambria Bold font (11780 bytes, 29 glyphs)
Found CJK font: /usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc
Font subset (cjk): OK 53 mappings, 53 widths
PostScript name: NotoSansCJKjp-Regular
Embedding CJK font (11840 bytes, 53 glyph mappings)
Found CJK Bold font: /usr/share/fonts/opentype/noto/NotoSansCJK-Bold.ttc
Font subset (Bold): OK 53 mappings, 53 widths
Embedding CJK Bold font (11926 bytes, 53 glyph mappings)
```

手順書の表でいうと、出たのは「同梱代替が効いた＝正常」の行と、`Symbol` に対する「代替が無い」警告のみ。代替が無いと報告された族は `Symbol` ただ一つ。

PDF の中身 — これが本題

	Type0	Type1	BaseFont
土台 basic_test	1	2	OxiLatin-Regular , OxiLatin-Bold , NotoSansCJKjp-Regular
ブランチ basic_test	4	0	Carlito-Regular , Carlito-Bold , Carlito-Italic , NotoSansCJKjp-Regular
土台 docs/sample	3	3	OxiCambria-Regular/Bold , OxiLatin-Bold , ...
ブランチ docs/sample	7	0	Caladea-Regular/Bold/Italic , Carlito-Bold , ...

土台の Type1 は /FontDescriptor を持たない、つまり**ファイルを伴わない裸の Type1**だった。狙いどおり消えている。BaseFont も OxiLatin-* という架空名から実フォント名に変わった。

W arrays が Type0 と同数にならない件 — 欠陥ではない

手順書は「W arrays が Type0 と同数」を期待しているが、Linux では和文を含む文書で必ず1本ずれる。追ったところ `writer.rs:279` が `.filter(|(_, w)| *w != 1000)` で幅 1000 のグリフを既定値として捨てている。全角和文はちょうど 1000 なので、和文だけの文書では `width_entries` が空になり /W が出ない。/DW 1000 が全グリフを正しく覆うので、これで正しい。

和文のみ・太字なしの最小 docx で確認:

```
obj 7 NotoSansCJKjp-Regular /W=False  - /DW 1000 のみ。全角なので正しい
```

土台でも同じ挙動。手順書 §4 の判定基準を「Type1 が 0」だけにするのが正しい。

4. 退路 (fontTools 無し) — 効く。ただし代償は Windows 実測の倍以上

basic_test.docx 1冊での比較:

条件	PDF サイズ	Type1	所要
土台	13,243 B	2	1.7 s
ブランチ + fontTools	32,996 B	0	4.2 s
ブランチ + python3 あり / fontTools 無し	14,026,080 B	0	—
ブランチ + python3 無し	14,026,080 B	0	1.6 s

両方の退路がバイト単位で同じ大きさに着地し、構造も健全 (Type1 0、実フォント名)。手順書の想定どおり動いている。

ただし **14 MB** は手順書が Windows で見た 6.09 MB の倍以上ある。理由は

```
Embedding cjk whole (15458582 bytes) - no subsetter available
```

丸ごと埋め込みの対象に **Noto CJK 全体 (15.4 MB)** が入るため。英語主体の文書でも CJK フォントは無条件に埋め込まれるので、Linux では退路のコストが Windows より一桁重い。素のコンテナで運用するなら fontTools は実質必

須。

なお CJK 太字だけは退路が無い。 `find_and_subset_cjk_font_bold` は失敗時に `return None` するだけで丸ごと埋め込みへ落ちない (`Bold font subsetting failed` の後、太字和文は消える)。

5. 一時ファイル — 消えない。さらに `/tmp` 直書きが1箇所残っている

`std::env::temp_dir()` 化そのものは効いており、Linux で正しく `/tmp` に置かれ、 `oxi-font-<tag>--<pid>-*` と `pid` 付きの名前になる。しかし:

① 誰も消さない。 `main.rs` に `remove_file` が1つも無い。30冊回した後:

```
$ ls /tmp/oxi-font-* | wc -l
372
$ du -ch /tmp/oxi-font-* | tail -1
2.1M    total
```

② `temp_dir()` 化が半分しか適用されていない。

```
土台 8ef96b91:
2432: let subset_path = format!("/tmp/oxi-font-subset-{}.otf", label);  ← 直った
2657: let subset_path = "/tmp/oxi-font-subset-bold.otf";                ← 残っている

ブランチ dbb06d08:
2733: let scratch = std::env::temp_dir();                                ← 直した側
2963: let subset_path = "/tmp/oxi-font-subset-bold.otf";                ← 手つかず
2964: let cidmap_path = "/tmp/oxi-font-cidmap-bold.json";
2965: let widths_path = "/tmp/oxi-font-widths-bold.json";
```

CJK 太字の経路だけ `/tmp` 直書きのまま、しかも `pid` が付かない固定名。同時に2本走らせると 互いのファイルを踏む。Linux では実害が見えにくいですが、**Windows ではこのパスは `C:\tmp\` を指すことになるはずで、そこが無ければ CJK 太字は黙って落ちる** (Windows では未検証。コードからの推定。英語コーパスでは踏まないのでも 0.9044 には影響していない)。

6. 予想していた壊れ方の答え合わせ

予想	結果
1. 日本語が出ない	外れ。日本語は普通に出る。CJK_FONT_PATHS には msgothic.ttc と並んで /usr/share/fonts/opentype/noto/NotoSansCJK-{Regular,Bold}.ttc が最初から入っている (main.rs:2363-2376)。cp -r ... fonts target/release/ は不要だった。pdftotext で「日本語のテスト文章です。幅の確認。」が正しく抽出でき、ToUnicode CMap も出ている
2. PDF が巨大	当たり。ただし桁が Windows より1つ大きい (6.09 MB → 14 MB)。上記 § 4
3. /tmp の一時ファイル	残る。372個 2.1 MB。さらに直書きが1箇所残存。上記 § 5
4. 索引が遅い	遅くない。2069 face で 0.73 秒。ただし毎プロセス払うので、1冊 3.1 秒のうち約1/4を占める

7. 48冊バッチ (§ 6) — コーパスが無いので代替した

pipeline_data/docx_corpus/en/ はこの機械に無い (pipeline_data/ は CSV と JSON のみ)。真値 PDF も無いので SSIM は当然出せない。代わりにリポジトリ内の fixture 30冊で 土台と ブランチを同条件で回した。

	成功	所要	Type1 合計	Type1 が残る冊数	PDF 合計
土台 8ef96b91	30/30	52 s	11	8 / 30	141.4 KB
ブランチ dbb06d08	30/30	93 s	0	0 / 30	273.3 KB

落ちた文書は無し。警告は Symbol の1件のみ。サイズは 1.93 倍、時間は 1.8 倍。

コーパスを送ってもらえれば § 6 のとおり回して oxi-linux-pdfs.tgz を返す。pipeline_data/docx_corpus/en/ をこの機械に置くだけでよい。

まとめ — 手順書の3つの問いへの答え

1. 建つか → 建つ。テストも全緑 (58 + 13)。Linux 固有の修正は要らなかった
2. フォントを見つけるか → 見つける。2069 face、0.73 秒、同梱16本すべて解決
3. Python 無しで PDF が正しく出るか → 出る。Type1 は 0 のまま。ただし 14 MB

土台で 30冊中 8冊に残っていた未埋め込みフォントが 0 になった。Linux では **この変更の主目的は達成されている**。SSIM は Windows 側でしか出せないので、コーパスが届き次第レンダして返す。

直す価値がありそうな3点

1. main.rs:2963-2965 の /tmp 直書きを temp_dir() + pid 付き名に揃える (Windows で CJK 太字が落ちる推定原因)
2. 一時ファイルの後始末が無い (372個 2.1 MB / 30冊)
3. env! ("CARGO_MANIFEST_DIR") がビルド時の絶対パスとして焼き込まれ、配布バイナリがビルドマシンのディレクトリを実行時に読む

2回目 — 修正版 51355a7a の実測 (2026-09-02)

feat/pdf-font-resolution が dbb06d08 → 51355a7a に進んだので同じ機械で測り直した。差分に含まれる font 関連は 2 commit (78706568 コレクション内の2番目以降の face、 51355a7a 東アジア文字の判定)。他は VBA / pptx で今回の対象外。

前回指摘した3点 — すべて直っている

	状態
① /tmp 直書き	解消。main.rs から "/tmp/oxi-font-*" が消え、temp_dir() 一本に統一。CJK 太字の 144 行のコピーも subset_font_file に統合された
② 後始末が無い	入った。ただし成功経路のみ (後述)
③ CARGO_MANIFEST_DIR	解消。実行ファイルがソースツリー内にあるときだけ使う判定が入った

③ は3形態すべてで意図どおり:

```
ソースツリー内      → .../crates/oxidocs-cli/fonts/Carlito-Regular.ttf#0
バイナリだけ移動    → /usr/share/fonts/truetype/crosextra/Carlito-Regular.ttf#0
fonts/ を隣に同梱    → <配布先>/fonts/Carlito-Regular.ttf#0
```

副次的に直ったもの

- CJK 太字の PostScript 名が出るようになった。以前は ps_name: None で NotoSansCJKjp-Regular を名乗っていたのが NotoSansCJKjp-Bold になった
- CJK 太字にも退路ができた。以前は subsetter 失敗時に return None して太字和文が消えていたが、Embedding cjk-bold whole (16023217 bytes) が出るようになった

needs_cjk の修正 — 効いている

純英語+スマートクォート+アクセント+タブの docx を作って確認:

```
It's a café – an en dash, a curly quote, and<TAB>a tab. Naïve résumé.
```

	CJK フォント	PDF
fontTools あり	引き込まない	11,481 B
fontTools 無し	引き込まない	272,135 B

Found CJK font: の行が一切出ない。退路でも 272 KB に収まる。

前回の私の報告を1つ訂正する

前回「英語主体の文書でも CJK フォントは無条件に埋め込まれる」と書き、14 MB の例として

tests/fixtures/basic_test.docx を挙げた。この fixture は実際に日本語を24文字含んでいる（日本語テスト：禁則処理の確認。「括弧」や、句読点。）。あの 14 MB は和文を含む文書の正当なコストで、c > 0x7F バグの例としては不適切だった。バグ自体は実在し、上の純英語 docx で修正が効いていることを確認した。

前回の「PDF 合計 141.4 KB → 273.3 KB」も du が .err ログを含めていた。PDF だけを数えた正しい値は下表のとおり。

30冊バッチ — 3世代の比較

	成功	所要	Type1	未埋め込みを含む冊数	PDF 合計
土台 8ef96b91	30/30	52 s	11	8/30 (OxiLatin-*, OxiCambria-*)	130.5 KB
修正前 dbb06d08	30/30	93 s	0	0/30	224.0 KB
修正後 51355a7a	30/30	94 s	0	1/30 (Symbol)	201.7 KB

ビルド 1m04s / rc=0 / error0。cargo test -p oxipdf-core は 58 + 13 passed、新規3本も緑のまま。

見つかった問題 2件

A. 本文の無い文書で subsetter が落ち、一時ファイルが残る

30冊のうち 22冊（すべて tests/fixtures/minimal_repro/ 配下の、テキストを持たない表だけの文書）で、Latin 系4面すべての subsetting が落ちる：

```
Font subsetting (latin) failed: Traceback (most recent call last):
  File "<string>", line 32, in <module>
    for unicode_val, glyph_name in cmap.items():
AttributeError: 'NoneType' object has no attribute 'items'

Embedding latin whole (628032 bytes) – no subsetter available
```

main.rs:2845 の cmap = font.getBestCmap() が、文字集合が空のまま subset された フォントに対して None を返す。これが latin / latin-bold / cambria / cambria-bold の4回起きる。

これはブランチで新たに踏むようになった経路。土台 8ef96b91 では同じ文書に Warning: No Latin system font found が出るだけで、subsetter を呼んでいなかった。Latin 面を見つけられるようになった結果、本文ゼロの文書にも subsetter を掛けている。

結果として ② の後始末が空振りする。font.save() (main.rs:2843) はクラッシュ前に .otf を書き終えているのに、失敗経路の return embed_whole_font(...) (main.rs:2911) が掃除ブロック (main.rs:2946-2953) より手前にあるため通らない：

```
$ ls /tmp/oxi-font-* | wc -l      # 30冊バッチ後
88                               # = 22冊 × 4面。すべて .otf (json 類は消えている)
```

本文のある文書では残骸ゼロなので、掃除そのものは正しく動いている。早期 return 3箇所のうち、`.otf` が書かれ得るのは `!output.status.success()` の1箇所だけ。

PDF の出力自体は壊れていない (`1row_none.docx` → 2,648 B)。丸ごと埋め込んだ 1.4 MB 分は使われずに捨てられるので、無駄仕事とゴミが残るだけ。

B. `Symbol` が `FontFile` を持たない `Identity-H` で出る

`comprehensive_test.docx` 1冊のみ。警告自体は前から出ている:

```
Warning: no font file for Symbol - these are drawn with a viewer substitute and will not match Word
```

PDF 側の形が変わった:

```
修正前 dbb06d08: Symbol の文字は NotoSansCJKjp-Regular で描かれていた (埋め込みあり)
修正後 51355a7a: << /Subtype /Type0 /BaseFont /Symbol /Encoding /Identity-H ... >>
                  << /Subtype /CIDFontType2 /BaseFont /Symbol /DW 1000 >>   ← FontFile なし
```

`needs_cjk` が正しくなった副作用。Symbol の文字は東アジア文字ではないので、もう CJK フォントへ流れず、ファイルの無い Symbol 面に残る。

どちらも正しい字形は出ないが、**Identity-H でファイルが無いのは前より脆い**。コンテンツストリームに入っているのは生のグリフ番号で、それを解釈すべきフォントが PDF に無い。ビューアが何を代替に選んでも、その字形表の別の場所を引く。

なお「`Type1` が 0」という判定ではこれを捕まえられない。**`Type1` が 0 かつ、すべての CIDFont が `FontFile` を持つ**が正しい判定基準。

まとめ

指摘した3点は直っている。主目的 (未埋め込みフォントの排除) も土台の 8/30 → 実質 0 で達成。残るのは上の A と B で、どちらも**この変更が新しく踏むようになった経路**にある。A は掃除ブロックを早期 return より前に出すか、空の文字集合を subsetter に渡さないかで消える。B は Symbol / Wingdings のような記号フォントに同梱の代替を用意するか、ファイルが無い面は Type0 ではなく標準14書体の Symbol として出すか、の判断待ち。

3回目 — blindC50 コーパス 50冊 (5288e673、2026-09-02)

oxi-blindC50-corpus.tgz を pipeline_data/ に展開し、README のとおり直列で回した。コードには一切手を入れていない（盲検セットなので当然だが、念のため）。

ビルド 1m00s / rc=0 / error0。cargo test -p oxipdf-core は 59 + 13 passed。

返すもの

- oxi-linux-pdfs.tgz — out_pdf/ 50本 + out_log/ 50本 (fontTools あり、これが本命)
- oxi-linux-pdfs-nofonttools.tgz — out_pdf_noft/ + out_log_noft/ (fontTools 無し)

ファイル名は指定どおり {種別}__{stem}.pdf。

数字

	冊数	所要	PDF 合計	一時ファイル残
fontTools あり (4.63.0)	50/50	143 s	8.21 MB	0
fontTools 無し	50/50	74 s	36.72 MB	0

総頁数 650。落ちた文書はゼロ。python3 -c "import fontTools" は通る (4.63.0)。

前回指摘した一時ファイルの残骸は完全に解消 (chars.is_empty() ガードと clear_scratch の全経路呼び出しが入ったため)。50冊 × 2条件を回して残骸 0。

判定スクリプト — 期待 0 件に対して 30 件

```
30 / 50 fail
('administrative__002dcbed4e04b487.pdf', 2, 3, 3)
('administrative__002e3a848b0f13d3.pdf', 1, 0, 0)
('administrative__003381c1f821ba4a.pdf', 1, 1, 1)
('administrative__003381e4f55ef08b.pdf', 2, 5, 5)
('correspondence__00481561b2898528.pdf', 1, 4, 4)
('correspondence__005243975f00b605.pdf', 1, 7, 7)
('creative__019f31375d4add14.pdf', 2, 0, 0)
('creative__01ac9ed7c1f1629b.pdf', 1, 0, 0)
('educational__002a301d7c46ba6e.pdf', 3, 5, 5)
('educational__002c8a33698eb867.pdf', 2, 1, 1)
('educational__002e0d67faa797b5.pdf', 1, 0, 0)
('forms__002f81ab0339a6c5.pdf', 3, 5, 5)
('forms__002fbe2c6e5f24b5.pdf', 1, 2, 2)
('forms__00396deef5c4871e.pdf', 1, 2, 2)
('forms__003a6fc626c84542.pdf', 3, 0, 0)
('legal__0019967c9e1c5ecf.pdf', 3, 8, 8)
```

```
( 'legal__001a2c7f07cd358f.pdf', 2, 3, 3)
( 'policies__003ccc9549f788e5.pdf', 1, 5, 5)
( 'policies__003e3c66a0ab5d8e.pdf', 2, 2, 2)
( 'policies__0040da3f99043e09.pdf', 2, 0, 0)
( 'policies__0046990ebc6e54df.pdf', 1, 2, 2)
( 'reference__0061531a57c4ac20.pdf', 2, 5, 5)
( 'reference__00643563d1d1f73d.pdf', 3, 5, 5)
( 'reference__0066cc3479a8a488.pdf', 1, 0, 0)
( 'reference__0068cf064981213b.pdf', 1, 1, 1)
( 'reference__0069c0f7e79b3448.pdf', 2, 5, 5)
( 'reports__00352896d2e2050a.pdf', 1, 2, 2)
( 'technical__007b1621e40d7649.pdf', 2, 3, 3)
( 'technical__008ae1fa42620401.pdf', 2, 5, 5)
( 'technical__00a54bff7e15af78.pdf', 3, 7, 7)
```

落ちている原因は全件 **Type1 != 0** のほう。3列目と4列目（CIDFontType2 と FontFile2）は30件すべてで一致しており、複合フォント側は50/50で健全。

種類の偏りは無い:

```
administrative 4/5  correspondence 2/5  creative 2/5  educational 3/5  forms 4/5
legal 2/5  policies 4/5  reference 5/5  reports 1/5  technical 3/5
```

未埋め込みで残った 53 面の内訳

族名	面数	描画回数
Symbol	14	234
OxiCJK-Regular	8	83
OxiCJK-Bold	7	544
Wingdings	4	301
Verdana	4	3,745
Arial Narrow	3	2,874
Tahoma	2	27
Aptos / Arial Black / Calibri Light / Cambria Math / Century Gothic / Comic Sans MS / Garamond / Helvetica Neue / Segoe UI / Trebuchet MS / Trebuchet MS Bold	各1	—

これは飾りではない — 53面すべてが実際に描画に使われている

各ページの /Resources を（間接参照を辿って）解決し、コンテンツストリームの Tf オペレータが指す資源だけを数えた結果:

描画に使用 = 53 面

未参照 = 0 面

Verdana で 3,745 回、Arial Narrow で 2,874 回の描画がある。その2文書は本文がまるごと、PDF に入っていないフォントで描かれている。

3つの原因に分かれる

(1) 記号フォント — Symbol 14面 / Wingdings 4面

描かれているのはほぼ箇条書きの行頭記号:

```
Symbol      : U+F0B7 が 220回 (Word の Symbol 箇条書き)、■ 10回
Wingdings   : ■ 294回、▪ 4回、▶ 3回
```

同梱に代替が無い族。stderr で Warning: no font file for X として申告されている。

(2) OxijCjk-* 15面 — 東アジア文字を含まない文書に出る

administrative__002dcbed4e04b487.docx は <w:t> に非ASCII が1文字も無いのに OxijCjk-Regular と OxijCjk-Bold が Type1 で出て、実際に使われている。描いている中身は:

```
OxijCjk-Bold   : U+25CF(●) 19回、U+2011(-) 、空白 220回、"of" 18回 ...
OxijCjk-Regular : U+2011(-) 64回、U+25CF(●) 12回、★ 4回
```

行頭記号の ● と、非分割ハイフン U+2011 が起点。needs_cjk は latin_face がある場合「その面の cmap に無い文字」を CJK 送りにするので、Carlito に無い U+2011 で CJK 面に切り替わり、その後の普通の英文(空白や "of")まで一緒に持っていかれている。

15面のうち1面は別口で、is_cjk_family (main.rs:2460) が

```
|| base.contains("Gothic") || base.contains("Mincho") || ...
```

と部分一致で判定しているため Century Gothic が東アジアの族として扱われている。

(3) 代替の無い実在の族 — 20面

Verdana Tahoma Segoe UI Comic Sans MS Garamond Helvetica Neue Aptos と、族名の変種 Arial Narrow Arial Black Calibri Light Cambria Math Trebuchet MS Bold。

5288e673 (「代替が合うと分かっていない族はそのままにする」) の方針どおりの結果だが、判定基準の Type1 == 0 とは両立しない。どちらを取るかは判断が要る。

なお変種の扱いには余地がある。Arial Narrow はこの機械に

/usr/share/fonts/truetype/liberation/LiberationSansNarrow-*.ttf が実在するし、Calibri Light / Trebuchet MS Bold は「族名+ウェイト」が族名として来ているので、分解すれば既存の代替に当たる。

fontTools 無しとの差 — 1文書だけ結果が変わる

```
fontTools あり : 30 / 50 fail
fontTools 無し : 29 / 50 fail
```

差分は `technical__007b1621e40d7649.pdf` の1件のみ。

```
ft あり にのみ存在: ('Type1', 'OxiCJK-Bold'), ('Type1', 'OxiCJK-Regular')
ft 無し にのみ存在: なし
```

subset 経路だと CJK 面が未埋め込みで残り、丸ごと埋め込みの退路だと埋め込まれる。 README は「subset の有無で見た目は変わらないはず」としていたので、これが該当する差。他の 49 文書はフォント構成が完全に一致した。

残っている小さいもの

`getBestCmap()` の `None` はまだ 2 文書で出る (`FNotoSansSymbols` / `FNotoSansSymbols_B`)。 `chars.is_empty()` のガードは「そもそも文字が無い」場合しか塞いでいない。 **要求した文字がその面に1つも無いと**、subset 後に `cmap` が消えて同じ例外になる。一時ファイルは `clear_scratch` が拾うので残らず、退路も効くので出力は壊れていない。

`subset_font_file` の `child.wait_with_output().ok()`? (`main.rs:2938`) だけは `clear_scratch` を通らずに `None` を返す。 `font.save()` の後なので理屈上は残骸が出るが、 `wait` 自体が失敗する状況なので実害はほぼ無い。

4回目 — FONT_CHECKS.md の手順で (88d13003、2026-09-03)

crates/oxidocs-cli/FONT_CHECKS.md が新設されたので、その手順どおりに回した。前回から font 関連で4 commit 入っている。

```
88d13003 Say how to check that a page carries the fonts it draws with
fddb129 Point Arial Narrow at the face that measures the same, without shipping it
555d8d0c Refuse a subset that kept none of the glyphs it was asked for
60dc528d Read the cmap a symbol font actually carries, and name no font we did not load
```

手順書について

判定が「Type1 が 0」から**3つ**に変わった。とくに **ink 検査** (150dpi でラスタ化し、墨のある行数を数える) が加わったのが大きい。構造が正しくても字が出ていない —— 「要求したグリフを1つも残さなかった subset」 —— を、構造検査だけでは捕まえられる、というのはそのとおり。前回まで私が使っていた基準では見えない欠陥だった。

type1 の位置づけも変わっている。「0 が目標だが、コードを変えて 0 にするのは違う。誰も持っていないフォントを名指した文書は、合わないもので描かない限り 0 にできず、それは黙って合わないものを描くより悪い」 —— この整理に異論はない。前回私が「Type1 == 0 と両立しない」と書いた点は、この手順書の立場で解決している。

なお fitz は PyMuPDF 1.28 で非推奨になっており、手順書のスクリプトは DeprecationWarning を出す (import pymupdf が新しい綴り)。動作には影響しない。この機械には PyMuPDF が入っていなかったので venv に入れて回した。

ビルドと試験

```
cargo build --release -p oxidocs-cli → Finished in 1m 01s / rc=0 / error 0
cargo test -p oxipdf-core → 59 passed + 13 passed / 0 failed
cargo test -p oxidocs-cli → 2 passed / 0 failed
```

手順書が名指しする不変条件の試験も緑:

```
test writer::tests::every_composite_font_carries_its_font_file ... ok
```

blindC50 50冊 — 3つの検査

検査	結果	判定
composite_ok (CIDFontType2 == FontFile2)	50/50 で成立	不変条件を満たす
ink_rows == 0 (白紙)	0 冊	合格
type1 != 0	24 冊 (前回 30 冊)	この機械に無い族

ink_rows 合計 528,125。50/50 成功、148 s、PDF 合計 8.87 MB、一時ファイル残 0。

前回からの変化 — 悪化ゼロ、14冊が改善

旧版（5288e673）の PDF と1冊ずつ突き合わせた。

type1 が減った文書: 14 type1 が増えた文書: 0
ink 合計 527,891 → 528,125

未埋め込みの面数は **53 → 33**。消えた20面の内訳:

	前回	今回	効いた commit
OxiCJK-Regular / OxiCJK-Bold	15	0	60dc528d
Arial Narrow	3	0	fddb129
Arial Black	1	0	60dc528d (族名の接尾語分解)
Calibri Light	1	0	同上

OxiCJK-* は行き先が変わった。前回「東アジア文字が無いのに CJK 面に送られ、その後の英文まで持っていられる」と書いた文書は:

旧 5288e673: Type1:OxiCJK-Bold, Type1:OxiCJK-Regular ← 未埋め込み
新 88d13003: Type0:NotoSansSymbols-Medium, -SemiBold ← 埋め込み済み

行頭記号の ● と非分割ハイフン U+2011 が、実在する記号フォントに載って埋め込まれている。

ink が 2.4% 減った1冊は改善だった

policies_003e3c66a0ab5d8e.pdf だけ ink が 1229 → 1199 に減ったので中を見た。減った理由は字が消えたのではなく、壊れていた字が直ったため:

旧: Head Start, EarlyHead Start, ... / What are the components of a Head Start program?
新: Head Start, Early Head Start, ... / What are the components of a Head Start program?
旧: socially,and emotionally. → 新: socially, and emotionally.

Calibri Light が未埋め込み (Type1) だったのが Carlito-Regular の埋め込みに変わり、余計な ' と詰まった空白が消えた。頁数は2頁で変わらない。

前回の指摘の後始末

getBestCmap() の None 例外は **50冊のログすべてで 0 件**。555d8d0c により、正常系のメッセージに変わった:

Font subset (X) mapped no glyph; embedding the whole font instead

発生は4回 (FNotoSansSymbols ×2、FNotoSansSymbols_B、FArialBlack)。手順書の表にも「normal」として載っている。

残っている 33 面

族名	面数
Symbol	14
Wingdings	4
Verdana	4
Tahoma	2
Aptos / Cambria Math / Century Gothic / Comic Sans MS / Garamond / Helvetica Neue / Segoe UI / Trebuchet MS / Trebuchet MS Bold	各1

すべて `Warning: no font file for X` として `stderr` に申告されている。手順書の立場では、これはコードで潰すものではなく、その族を入れるか、実測で `advance widths` が一致する代替を足すかの話。

`Century Gothic` は前回 `is_cjk_family` の `contains("Gothic")` に引っかかって CJK 面に送られていたが、今回は普通の未対応族として扱われている。`Trebuchet MS Bold` が `Trebuchet MS` と並んで残るのは、接尾語分解は効いたが `Trebuchet MS` 自体がこの機械に無く代替も無いため。手順書どおりの挙動。

返すもの

`oxi-linux-pdfs-88d13003.tgz` — `out_pdf/` 50本 + `out_log/` 50本。ファイル名は `{種別}__{stem}.pdf`。

フォント解決の macOS 実測

crates/oxidocs-cli/FONT_CHECKS.md の手順を macOS 実機で通した記録。本書の前半と同じ先端・同じ blindC50 50冊を、フォント事情が正反対の機械で回したもの。

SSIM の採点は含まない（真値が Word の PDF なので Windows が要る）。ここにあるのはレンダ結果と、その構造・ink の検査。

項目	値
OS	macOS 26.5.2 (25F84) / arm64
cargo	1.96.0
python3	3.9.6 (Xcode 同梱)
fontTools / PyMuPDF	既定では両方とも無い。検査のため venv に導入 (fontTools 4.60.2)
索引された face	740
測ったもの	feat/pdf-font-resolution の 88d13003

Linux とちょうど裏返しの機械。Microsoft のコア書体が実物で入っている一方、Office 由来の Calibri / Cambria / Segoe UI は無い。

ビルドと試験

```
cargo build --release -p oxidocs-cli    → Finished in 1m 01s / rc=0 / error 0
cargo test  -p oxipdf-core              → 59 passed + 13 passed / 0 failed
cargo test  -p oxidocs-cli              → 2 passed / 0 failed
test writer::tests::every_composite_font_carries_its_font_file ... ok
```

Linux 固有・macOS 固有の分岐は踏んでいない。どちらでも同じ数の試験が通る。

索引 — 740 face、0.57 秒

既定クエリ 11 族 44 行の内訳は、解決 28 / DEGRADED 8 / ABSENT 8。

実物が見つかる族:

Arial	/System/Library/Fonts/Supplemental/Arial.ttf#0	(4スタイル揃い)
Times New Roman	.../Supplemental/Times New Roman.ttf#0	(4スタイル揃い)
Courier New	.../Supplemental/Courier New.ttf#0	(4スタイル揃い)
Verdana	.../Supplemental/Verdana.ttf#0	(4スタイル揃い)
Georgia	.../Supplemental/Georgia.ttf#0	(4スタイル揃い)
Tahoma	.../Supplemental/Tahoma.ttf#0	(regular / bold のみ)
Wingdings	.../Supplemental/Wingdings.ttf#0	(regular のみ)
Symbol	/System/Library/Fonts/Symbol.ttf#0	(regular のみ)

ABSENT は Calibri / Cambria / Segoe UI の3族。前2つは同梱の Carlito / Caladea が受ける。

DEGRADED が出るのはこの機械

Tahoma italic や Wingdings bold のように、族はあるがそのスタイルが無い場合に出る:

```
Tahoma      italic      DEGRADED -> .../Supplemental/Tahoma.ttf (regular)
Wingdings    bold        DEGRADED -> .../Supplemental/Wingdings.ttf (regular)
```

Linux では該当が無く、この出力形は macOS で初めて見えた。

コレクション

システムに .ttc が 128 本ある。Helvetica / Courier / Hiragino Sans などはコレクションとして索引に載る:

```
Helvetica      /System/Library/Fonts/Helvetica.ttc#0
Courier         /System/Library/Fonts/Courier.ttc#0
Hiragino Sans   /System/Library/Fonts/ヒラギノ角ゴシック W8.ttc#0
```

ただし #1 以降に解決した族は今回は 0 件。コーパスが名指した族はすべて #0 (コレクションの先頭か、単体の .ttf) に当たった。resolve_face_path がコレクションの2番目以降を外す制限は、この機械では踏まない。

blindC50 50冊

	冊数	所要	PDF 合計	一時ファイル残
fontTools あり (4.60.2)	50/50	72 s	10.9 MB	0
fontTools 無し	50/50	27 s	61.8 MB	0

TMPDIR は macOS では /tmp ではなく /var/folders/.../T。std::env::temp_dir() 化がここで効いていて、実 TMPDIR に残骸は出ない。

3つの検査

検査	fontTools あり	fontTools 無し
composite_ok	50/50 成立	50/50 成立
ink_rows == 0	0 冊	0 冊
type1 != 0	7 冊	7 冊
ink_rows 合計	527,947	528,198

subset の有無で未埋め込みの族は1つも変わらない。手順書が「見た目は変わらないはず」と書いているとおりで、変わるのはサイズ (5.7倍) と所要時間だけ。

未埋め込みで残った 7 面

族名	面数
Noto Sans Symbols	2
Aptos / Cambria Math / Century Gothic / Garamond / Segoe UI	各1

すべて `Warning: no font file for X` として `stderr` に申告されている。この機械に実在しない族ばかりで、手順書の立場ではこれが正しい状態。

`Symbol` と `Wingdings` は実物があるので、記号フォントの経路 (`Font subset (X) mapped no glyph; embedding the whole font instead`) を通って丸ごと埋め込まれている——`FSymbol` 13回、`FSymbol_B` 2回、`FWingdings` 4回、`FArialBlack` 1回。

代替が使われたのは Calibri 36回、Cambria 6回、Calibri Light 2回。

Linux との突き合わせ

同じ 50 冊、同じ `88d13003`。

	Linux	macOS
索引された face	2,069	740
<code>type1 != 0</code> の冊数	24 / 50	7 / 50
未埋め込みの面数	33	7
<code>composite_ok</code>	50/50	50/50
白紙	0	0
総頁数	650	650
<code>ink_rows</code> 合計	528,125	527,947
PDF 合計 (subset 有)	8.87 MB	10.9 MB

頁数は 1 冊の違いもなく一致した。レイアウトはフォントの当たり外れに依らない。`ink` 合計の差も 0.03% で、最大の 1冊でも 331 行 (`technical__008ae1fa42620401`、Linux 17,896 / Mac 17,565)。実際に違う書体で描かれている以上、この程度の差は出る。

未埋め込みの族は補完関係にある。macOS は Linux で落ちた `Verdana` `Tahoma` `Arial Narrow` `Arial Black` `Trebuchet MS` `Comic Sans MS` `Helvetica Neue` `Symbol` `Wingdings` をすべて実物で解決する。逆に `Noto Sans Symbols` は Linux にあって macOS に無い。両方に無いのは `Aptos` `Cambria Math` `Century Gothic` `Garamond` `Segoe UI` の5族。

この機械で分かったこと

- 1. `DEGRADED` の経路は macOS でしか踏まない。族はあるがスタイルが無い、という形は Microsoft 書体を実物で持つ機械に固有
- 2. `TMPDIR` が `/tmp` でない環境でも残骸ゼロ。`std::env::temp_dir()` 化の効き目が Linux より分かりやすく出る場所

3. macOS には既定で fontTools も PyMuPDF も無い。手順書の検査を回すには venv などですす必要がある
(Xcode 同梱の python3 は 3.9.6)